Messaging Terminology

1. What is a Messaging System?

A **messaging system** transfers data between applications/services so they don’t need direct tight coupling.

```
None
Service A → Messaging System → Service B
```

Helps developers focus on business logic instead of communication complexity.

2. Why Messaging Systems Are Used

They help with:

```
None
Decoupling services

Async processing

Scalability

Retry handling

Load buffering

Fault tolerance

Event driven architecture
```

3. Message Queue

A **message queue** stores messages waiting to be processed.

```
None
Producer → Queue → Consumer
```

Each message is usually processed once by one consumer.

Example:

```
None
Order placed

Payment request

Email job

Image resize task
```

4. What is a Message?

A **message** is a unit of work/data.

```
None
{

 order_id: 101,

 action: "send_email"

}
```

5. Producer / Consumer

Also called:

```
None
Publisher / Subscriber

Pub/Sub
```

Meaning:

```
None
Producer = creates message

Consumer = processes message
```

6. Pub/Sub Model

```
None
Producer publishes event

Multiple subscribers consume event
```

Example:

```
User Signup Event
   ↓
Email Service
Analytics Service
CRM Service
```

7. Message Broker

A **message broker** manages delivery/routing of messages between systems.

Examples:

```
Apache Kafka
RabbitMQ
ActiveMQ
Amazon SQS
```

Responsibilities:

```
None
Queue storage

Routing

Retries

Acknowledgements

Protocol translation

Scaling consumers
```

8. Kafka vs RabbitMQ

```
None
Kafka     → High throughput event streaming

RabbitMQ  → Traditional queueing + routing
```

9. Event Streaming

Continuous flow of events processed in real time.

```
Clicks

Payments

Sensor data

Logs

Transactions
```

Architecture:

```
Producers → Kafka Topics → Consumers
```

10. What is an Event?

An event describes change of state.

```
Order Created

Payment Completed

User Logged In

Stock Price Changed
```

11. Pull vs Push

Pull Model

Consumer asks for data.

```
None
Consumer → Queue: give next message
```

Push Model

System sends data automatically.

```
None
Broker → Consumer directly
```

12. Why Pull is Often Better

Pull gives consumer control.

```
None
No overload

Backpressure handling

Rate control

Easy scaling

Safer for slow consumers
```

13. Example Pull System

```
Queue has 10,000 jobs

Consumer fetches 100 at a time

Processes safely
```

14. Why Push Can Be Better

Push useful when:

```
Need instant updates

Realtime notifications

Live video/audio

Webhook integrations

External providers sending data
```

15. Push Example

```
Payment Gateway → Webhook → Your API
```

16. Streaming Media Uses Push

For live audio/video:

```
None
Low latency more important than retry

Uses UDP often

Dropped packets may be ignored
```

17. Pull vs Push Summary

Feature	Pull	Push
Control	Consumer	Producer/Broker
Overload Risk	Lower	Higher
Latency	Slightly higher	Lower
Best For	Queues	Notifications/live streams

18. Real HLD Example

```
None
Order Service
   ↓
Kafka
   ↓
Inventory Service
Email Service
Analytics Service
Fraud Service
```

Loose coupling achieved.

19. Benefits in System Design

```
None
Independent deployments
Retry failed jobs
Scale consumers separately
Absorb traffic spikes
Async workflows
Better resilience
```

20. Common Problems

```
Duplicate messages

Ordering issues

Poison messages

Consumer lag

Dead letter queues needed

Exactly-once complexity
```

21. Interview Talking Points

If asked messaging architecture:

```
Use Kafka for streaming

Use RabbitMQ/SQS for task queues

Prefer pull consumers

Use DLQ for failures

Use partitions for scale

Idempotent consumers
```

22. One-Line Summary

Messaging systems enable scalable asynchronous communication between services using queues, brokers, producers, and consumers, while event streaming handles continuous real-time data flow efficiently.